

Italian Restaurant Sales Analytics

04 - Employees Data Cleaning and Preparation

Objective

This notebook cleans and validates the **Employees** worksheet from the Italian Restaurant Sales Analytics dataset. It verifies data quality, checks for missing values and duplicates, validates data types, and exports a cleaned dataset for further analysis in Python, SQL, and Power BI.

Step 1 - Import Libraries

The first step is to import the Python libraries required for data manipulation and analysis.

```
In [1]: # Import pandas for data manipulation and analysis  
import pandas as pd
```

Step 2 - Load the Excel Workbook

The next step is to define the location of the Excel file and load the workbook into Python.

```
In [2]: # Define the relative path to the Excel workbook  
file_path = "../data/raw/restaurant-sales-analytics-data.xlsx"
```

Step 3 - Read the Employees Worksheet

Load the Employees worksheet into a pandas DataFrame.

```
In [3]: # Load the Employees worksheet into a pandas DataFrame  
employees = pd.read_excel(file_path, sheet_name="Employees")
```

Step 4 - Preview the Dataset

Display the first five rows of the dataset.

```
In [4]: # Display the first five rows  
employees.head()
```

```
Out[4]:
```

	Employee	Quantity	Quantity (%)	Sales	Sales (%)
0	Employee 1	15786.200000	19.551131	61641.63	19.495142
1	Employee 2	14256.900000	17.657100	54125.31	17.117986
2	Employee 3	12638.866667	15.653174	45762.89	14.473239
3	Employee 4	11382.500000	14.097170	42810.66	13.539549
4	Employee 5	10697.500000	13.248801	40654.99	12.857784

Step 5 - Inspect the Dataset Structure

Display the structure of the dataset, including data types, number of non-null values, and memory usage.

```
In [5]: # Display dataset information
employees.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 17 entries, 0 to 16
Data columns (total 5 columns):
#   Column          Non-Null Count  Dtype
---  -
0   Employee        17 non-null    object
1   Quantity        17 non-null    float64
2   Quantity (%)    17 non-null    float64
3   Sales           17 non-null    float64
4   Sales (%)       17 non-null    float64
dtypes: float64(4), object(1)
memory usage: 812.0+ bytes
```

Step 6 - Check Dataset Dimensions

Display the number of rows and columns in the dataset.

```
In [6]: # Display dataset dimensions
employees.shape
```

```
Out[6]: (17, 5)
```

Step 7 - Check Missing Values

Identify whether the dataset contains any missing values in each column.

```
In [7]: # Check for missing values
employees.isnull().sum()
```

```
Out[7]: Employee      0
        Quantity      0
        Quantity (%)   0
        Sales          0
        Sales (%)      0
        dtype: int64
```

Step 8 - Check for Duplicate Rows

Identify whether the dataset contains duplicate records that may affect the analysis.

```
In [8]: # Check for duplicate rows
employees.duplicated().sum()
```

```
Out[8]: np.int64(0)
```

Step 9 - Display Column Names

Display all column names in the dataset.

```
In [9]: # Display column names
employees.columns
```

```
Out[9]: Index(['Employee', 'Quantity', 'Quantity (%)', 'Sales', 'Sales (%)'], dtype='object')
```

Step 10 - Display Summary Statistics

Display descriptive statistics for the numerical columns to better understand the distribution of the data.

```
In [10]: # Display summary statistics
employees.describe()
```

Out[10]:	Quantity	Quantity (%)	Sales	Sales (%)
count	17.000000	17.000000	17.000000	17.000000
mean	4748.597451	5.881114	18598.452941	5.882055
std	5748.721072	7.119763	21820.906375	6.901207
min	23.000000	0.028485	73.210000	0.023154
25%	331.000000	0.409942	1282.710000	0.405677
50%	1394.333333	1.726875	5343.320000	1.689910
75%	10697.500000	13.248801	40654.990000	12.857784
max	15786.200000	19.551131	61641.630000	19.495142

Step 11 - Check Unique Employees

Display the unique values in the Employees column to verify that all expected categories are present.

```
In [11]: # Display unique Employees
employees["Employee"].unique()
```

```
Out[11]: array(['Employee 1', 'Employee 2', 'Employee 3', 'Employee 4',
        'Employee 5', 'Employee 6', 'Employee 7', 'Employee 8',
        'Employee 9', 'Employee 10', 'Employee 11', 'Employee 12',
        'Employee 13', 'Employee 14', 'Employee 15', 'Employee 16',
        'Employee 17'], dtype=object)
```

Step 12 - Count Unique Employees

Count the number of unique Employees in the dataset.

```
In [12]: # Count unique Employees
employees["Employee"].nunique()
```

```
Out[12]: 17
```

Step 13 - Count Unique Quantities

Count the number of unique quantities in the dataset.

```
In [13]: # Count unique quantity
employees["Quantity"].nunique()
```

```
Out[13]: 17
```

Step 14 - Count Unique Sales Values

Count the number of unique Sales Values in the dataset.

```
In [14]: # Count unique sales values
employees["Sales"].nunique()
```

```
Out[14]: 17
```

Step 15 - Display Data Types

Display the data type of each column individually.

```
In [15]: # Display data types
employees.dtypes
```

```
Out[15]: Employee      object
Quantity    float64
Quantity (%) float64
Sales       float64
Sales (%)   float64
dtype: object
```

Step 16 - Check for Negative Sales

Verify whether the dataset contains any negative sales values.

```
In [16]: # Check for negative sales values
(employees["Sales"] < 0).sum()
```

```
Out[16]: np.int64(0)
```

Step 17 - Check for Zero Sales Values

Verify whether the dataset contains Sales Values equal to zero.

```
In [17]: # Check for zero sales values
(employees["Sales"] == 0).sum()
```

```
Out[17]: np.int64(0)
```

Step 18 - Check for Negative Quantities

Verify whether the dataset contains negative quantities.

```
In [18]: (employees["Quantity"] < 0).sum()
```

```
Out[18]: np.int64(0)
```

Step 19 - Check for Zero Quantities

Verify whether the dataset contains zero quantities.

```
In [19]: (employees["Quantity"] == 0).sum()
```

```
Out[19]: np.int64(0)
```

Step 20 - Create a Clean Copy

```
In [24]: # Reset the DataFrame index
employees_cleaned = employees.copy()
```

Step 21 - Preview the Clean Dataset

Display the first rows of the cleaned dataset.

```
In [21]: # Preview cleaned dataset
employees.head()
```

```
Out[21]:
```

	Employee	Quantity	Quantity (%)	Sales	Sales (%)
0	Employee 1	15786.200000	19.551131	61641.63	19.495142
1	Employee 2	14256.900000	17.657100	54125.31	17.117986
2	Employee 3	12638.866667	15.653174	45762.89	14.473239
3	Employee 4	11382.500000	14.097170	42810.66	13.539549
4	Employee 5	10697.500000	13.248801	40654.99	12.857784

Step 22 - Export the Clean Dataset

Export the cleaned dataset as a new Excel file for future analysis.

```
In [22]: # Export cleaned Employees dataset
employees.to_excel(
    "../data/cleaned/employees_cleaned.xlsx",
    index=False
)
```

Step 23 - Cleaning Summary

Display a summary of the data quality checks performed during the cleaning process.

```
In [23]: print("=====  
CLEANING SUMMARY  
=====")  
  
print(f"Rows: {employees.shape[0]}")  
print(f"Columns: {employees.shape[1]}")  
  
print(f"Missing values: {employees.isnull().sum().sum()}")  
print(f"Duplicate rows: {employees.duplicated().sum()}")  
  
print(f"Unique Employees: {employees['Employee'].nunique()}")  
  
print(f"Negative Sales: {(employees['Sales'] < 0).sum()}")  
print(f"Zero Sales: {(employees['Sales'] == 0).sum()}")  
  
print(f"Negative Quantity: {(employees['Quantity'] < 0).sum()}")  
print(f"Zero Quantity: {(employees['Quantity'] == 0).sum()}")  
  
print("Clean dataset exported successfully.")  
print("=====")
```

```
=====  
CLEANING SUMMARY  
=====  
Rows: 17  
Columns: 5  
Missing values: 0  
Duplicate rows: 0  
Unique Employees: 17  
Negative Sales: 0  
Zero Sales: 0  
Negative Quantity: 0  
Zero Quantity: 0  
Clean dataset exported successfully.  
=====
```

Author

Guido Acosta

Data Analyst Portfolio Project

GitHub Portfolio – 2026